SafeExec: A Hardened, Threat-Modeled Python Execution Sandbox for LLM-Agent Tool-Use

> ***AI-use disclosure.*** *Drafted with Codex (GPT-5, Codex desktop app) from student-authored project artifacts, command outputs, prior Canvas packages, and repository evidence. AI-drafted, student-revised. Full audit trail:* `docs/ai-use-log.md`.

Student: Zixuan Liang

Program: M.S. Computer Information Sciences, Harrisburg University

Course: CISC 699-50-A-2026/Summer - Applied Project in Computer Information Sciences

Advisor: Prof. Khalid Lateef

Instructor: Dr. Majid Shaalan

Final package date: 2026-06-27

Repository: `https://github.com/liangzixuan/cisc-699`

Abstract

Large language model agents increasingly execute generated code as part of tool-use workflows. This creates a systems-security boundary where generated text becomes operating-system process execution. SafeExec is a compact, reproducible Python execution sandbox designed to evaluate that boundary rather than merely demonstrate it. The artifact provides a synchronous execution API, a development local-subprocess backend, a hardened Docker backend, and a gVisor runtime path through Docker's `runsc` runtime. The project ties implementation to a threat model, functional tests, containment-oriented probes, benchmark timing data, environment snapshots, reproducibility checks, and explicit issue tracking.

The final integrated package shows a runnable SafeExec artifact with source code, setup instructions, smoke and unit tests, a local/API validation workflow, reproducibility audit, clean package evidence, Docker/gVisor target-host benchmark evidence, and a presentation/demo package. Fresh final evidence passed smoke, unit tests, local validation/API trace, and reproducibility audit.

The strongest target-host comparison generated 130 records: local 30/30, hardened Docker 50/50, and gVisor 49/50. The most important finding is positive but bounded: the measurement path works, Docker is stable on the current probe set, and gVisor is integrated, but final performance claims must distinguish cold-start overhead from program runtime. The project therefore contributes a transparent, evidence-oriented sandbox artifact and a clear record of remaining work around larger corpus coverage.

1. Problem Statement and Significance

LLM agents are useful because they can plan, call tools, interpret results, and continue working. They are risky for the same reason. When an agent generates or modifies code and sends it to an execution tool, natural-language uncertainty crosses into process execution. Prompt injection, excessive agency, or an ordinary model error can become file access, network egress, resource exhaustion, or host-environment exposure if the executor is weak. SafeExec addresses one narrow version of that problem: Python code execution for LLM-agent tool use on a controlled Linux host. The project does not claim to solve prompt injection, production multitenancy, authentication, TLS, billing, or cloud operations. Instead, it asks whether a small academic artifact can make the code-execution boundary observable:

- What backend ran the code?
- Which limits were applied?
- Was output returned in a structured format?
- Were network, filesystem, and resource controls exercised?
- Could another reader reproduce the evidence?

This is significant because many sandbox discussions are product-oriented or architecture-oriented but do not expose the evaluation path. Managed systems such as OpenAI Code Interpreter, Anthropic code execution, E2B, and Modal Sandboxes show that sandboxed code execution is now a mainstream agent capability. SafeExec's contribution is smaller and more inspectable: a threat-modeled execution service plus reproducible evidence comparing hardened Docker and gVisor on the same request/result model.

2. Related Work and Background

The project is grounded in four source groups.

First, LLM security sources explain why tool-use can turn language-level risk into systems risk. NIST's adversarial machine learning taxonomy and OWASP's LLM Top 10 frame LLM application risks in terms of attacker goals, excessive agency, prompt injection, and unsafe downstream actions [1], [2]. Greshake et al. show how indirect prompt injection can compromise LLM-integrated applications through remote content [3]. These sources justify treating generated code as untrusted even when the visible user intent seems benign.

Second, Linux isolation and container-hardening sources shape the design. Seccomp-BPF, namespaces, cgroups, capabilities, non-root users, read-only root filesystems, and network namespace controls are complementary, not interchangeable [5]. Historical runc vulnerabilities such as CVE-2019-5736 and CVE-2024-21626 support the project's refusal to treat default Docker as a complete adversarial boundary [7], [8]. In SafeExec, Docker is useful only when its hardening choices are visible and tested.

Third, gVisor and Firecracker clarify the isolation-vs-overhead tradeoff. gVisor moves much of the system-call surface into a userspace Sentry, reducing direct host-kernel exposure at the cost of compatibility and overhead [4], [6]. Firecracker is outside this implementation scope, but its microVM model explains why stronger isolation may be attractive for future work [9].

Fourth, evaluation and reproducibility literature shapes the evidence plan. SWE-bench emphasizes execution-based evaluation rather than subjective claims [14]. HumanEval and MBPP provide functional Python task sources that can seed benign correctness testing [17], [18]. Reproducibility guidance from Pineau et al. supports capturing command lines, versions, sample counts, host metadata, and limitations [15], [16].

3. Requirements, Scope, and Success Criteria

SafeExec's approved scope is intentionally narrow:

- Python 3.11 code execution only.

- Single-tenant research artifact, not production service.
- No outbound network from sandboxed code.
- Ephemeral per-request workspace.
- Structured execution results with stdout, stderr, exit code, duration,

backend, containment flag, containment reason, and applied limits.

- Hardened Docker as the primary container baseline.
- gVisor through `runsc` as the stronger-isolation comparison path.
- Functional tests, containment-oriented probes, benchmark records, and

reproducibility evidence submitted as first-class artifacts.

The original success targets were intentionally ambitious: runnable API path, functional-correctness evidence, adversarial containment evidence, Docker/gVisor comparison, reproducibility, and transparent AI-use disclosure. The final artifact satisfies the runnable API, local validation, target-host Docker/gVisor comparison, reproducibility, documentation, and disclosure requirements. The largest remaining gap is not that the artifact is nonfunctional; it is that the full planned HumanEval/MBPP and adversarial-corpus expansion is not yet as deep as the original stretch target.

4. System Design

SafeExec is built around one request model and one result model. A client sends Python code and backend/limit choices. The service validates the request, selects a backend, executes the program, and returns a structured result.

```
Client or test harness
  -> ExecutionRequest(code, backend, limits)
    -> safeexec.service.execute_code()
      -> LocalSubprocessBackend | DockerBackend | DockerBackend(runtime="runsc")
        -> ExecutionResult(status, stdout, stderr, exit_code, duration, limits)
```

The local backend is explicitly a development shim. It uses a temporary directory and an isolated Python interpreter invocation (`python -I`), but it is not a security boundary. Its purpose is to let the API, result schema, smoke tests, and validation harness run on the macOS authoring machine.

The Docker backend constructs a hardened command with:

- `--network none`
- CPU, memory, and PID limits
- read-only root filesystem
- dropped Linux capabilities

- `no-new-privileges`
- tmpfs-mounted `/tmp`
- non-root user `65534:65534`
- `python:3.11-slim` as the runtime image

The gVisor backend reuses the same Docker command shape but adds `--runtime runsc`. This keeps the comparison methodologically clean: the same request, code, limits, image, and result schema are used, with only the runtime boundary changed.

5. Implementation Details

The implemented artifact is intentionally compact:

Area	File(s)	Responsibility
Request/result model	`src/safeexec/models.py`	Dataclasses for limits, requests, and execution results.
Backend routing	`src/safeexec/service.py`	Selects local, Docker, or gVisor backend from request.
Local backend	`src/safeexec/backends/local_subprocess.py`	Development-only temporary-directory subprocess executor.
Docker/gVisor backend	`src/safeexec/backends/docker.py`	Hardened Docker command builder and executor; gVisor uses `runtime="runsc"`.
API shell	`src/safeexec/api/server.py`	Minimal JSON `/health` and `/execute` HTTP server.
CLI	`src/safeexec/cli.py`, `src/safeexec/__main__.py`	Command-line interface for direct local execution.
Validation	`scripts/run_validation_workflow.py`	Repeatable local/API validation with JSON/text outputs.
Midpoint benchmark	`scripts/run_midpoint_evidence.py`	Correctness, failure-control, and containment probes across backends.
Reproducibility	`scripts/audit_reproducibility.py`, `scripts/package_artifact.py`	Audit and package paths.
Final evidence/package	`scripts/run_final_evidence.py`, `scripts/package_final_submission.py`	Final command transcripts and final artifact ZIP.

The code uses only the Python standard library for the core artifact. This keeps setup friction low and makes reviewer execution more predictable. The tradeoff is that the HTTP API is intentionally minimal rather than a production

FastAPI/Uvicorn service.

6. Evaluation Methodology

SafeExec's evaluation is layered:

1. **Smoke execution:** run a minimal program through the service layer and inspect structured output.
2. **Unit/functional tests:** verify local subprocess behavior, Docker command construction, API behavior, and validation workflow.
3. **Local/API validation workflow:** repeat deterministic local cases and a live local `/execute` API trace.
4. **Target-host Docker/gVisor benchmark:** run correctness, failure-control, and containment-oriented probes across local, Docker, and gVisor backends on the Linux target host.
5. **Reproducibility audit:** verify required docs, README setup markers, Make targets, dependency policy, and tool status.
6. **Clean package run:** build a source package, unpack it outside the working tree, and run smoke, tests, validation, and audit.
7. **Final integrated package:** submit the report, deck, evidence bundle, artifact package, release notes, and AI-use appendix together.

This methodology avoids the common failure mode of showing a screenshot of a successful output without controlled conditions. Each result table links to a command, output file, environment snapshot, or issue note.

7. Results

7.1 Fresh final local evidence

The final evidence bundle was generated with:

```
PYTHONPATH=src python3 scripts/run_final_evidence.py --output-dir docs/14-hard-stop-7/evidence --repeat 3
```

The command wrote outputs under `docs/14-hard-stop-7/evidence/`.

Check	Result	Evidence
Smoke execution	PASS	`evidence/smoke-output.txt`

Unit/functional tests	PASS	`evidence/unit-tests-output.txt`
Local validation workflow	PASS	`evidence/validation-summary.txt`
Reproducibility audit	PASS	`evidence/reproducibility-audit.md`

Local validation result:

```
include_docker: False
all_passed: True
local: 12/12 passed
api_trace: passed
```

The local validation cases exercise normal stdout, deterministic computation, stderr with non-zero exit code, timeout handling, and a live API trace.

7.2 Target-host Docker/gVisor evidence

The strongest current container evidence remains the target-host benchmark captured in the midpoint evidence package and copied into the final evidence folder:

Backend	Passed / Total	Pass rate	Mean duration	Median duration	Maximum duration
Local subprocess	30 / 30	100.0%	93.2 ms	32.9 ms	402.0 ms
Hardened Docker	50 / 50	100.0%	868.8 ms	771.7 ms	1985.4 ms
gVisor (`runsc`)	49 / 50	98.0%	1361.8 ms	1327.5 ms	3167.2 ms

Docker passed all current correctness, failure-control, and containment probes. gVisor passed all but one `tmpfs_write_allowed` run, which timed out at the current wall-time boundary. The interpretation is not "gVisor failed as a sandbox"; it is that timing methodology must distinguish container startup and program execution more carefully before stronger performance conclusions.

7.3 Reproducibility evidence

The reproducibility package added `.env.example`, `docs/reproducibility/`, `make repro-audit`, `make package-artifact`, and a clean-run transcript. The final package adds `make final-artifact` through `scripts/package_final_submission.py`.

Reproducibility artifact	Purpose

`.env.example`	Documents configuration knobs without secrets.
`docs/reproducibility/runbook.md`	Reviewer setup and execution path.
`docs/reproducibility/environment.md`	Host/toolchain assumptions.
`docs/reproducibility/data-and-redistribution.md`	Corpus and redistribution policy.
`scripts/audit_reproducibility.py`	Checks required docs, README markers, Make targets, and tool status.
`safeexec-final-artifact-package.zip`	Final source/evidence package for Canvas/reviewer handoff.

The final reproducibility audit passed with no blocking findings.

8. Analysis and Interpretation

The strongest positive signal is not a single pass/fail result; it is that the project now has a repeatable measurement path. The same request/result model is used by smoke tests, local validation, API traces, Docker/gVisor target-host probes, and the final evidence bundle. This validates the design decision to keep the execution service and evaluation harness close together.

Docker is the strongest backend result in the current evidence. The hardened command shape exercises non-root execution, network denial, read-only root filesystem, tmpfs-only write path, output truncation, and wall-time handling. Docker passed all current target-host records. This does not prove Docker is safe in a general sense; it shows the current hardening bundle behaves correctly on the submitted probe set.

gVisor is integrated and mostly passing. Its 49/50 result supports the claim that the runsc path works, but the one timeout also shows that performance methodology matters. For short agent-generated programs, startup and runtime are both user-visible, but they should be reported separately if the report claims anything about overhead. This is the clearest technical adjustment for future work.

The largest limitation is corpus depth. The project planned HumanEval/MBPP subsets and a larger adversarial suite. The final integrated package includes seeded functional and containment-oriented evidence, but it does not claim that the full original stretch targets are complete. This is reported transparently

instead of hidden.

9. Ethics, Security, Privacy, and Broader Impact

SafeExec is a dual-use security artifact. Its purpose is defensive: reduce the risk of LLM-agent code execution by making isolation behavior measurable. The adversarial suite should avoid copying live exploit code and should frame each program as a contained probe with expected outcomes.

The project uses no personal, FERPA-protected, HIPAA-regulated, proprietary, or confidential data. The current validation programs are student-authored. Future HumanEval/MBPP reuse must preserve upstream license and provenance notes.

The project should not be presented as a production sandbox. It lacks production authentication, authorization, TLS, tenant isolation, billing, observability, incident response, and long-term patch management. Its value is as an academic and portfolio artifact that demonstrates disciplined systems thinking, reproducibility, security reasoning, and honest evidence reporting.

AI assistance was used for drafting, code review support, documentation structure, evidence interpretation, and implementation help. The student remains responsible for correctness, final wording, and all submitted artifacts. The final AI-use appendix is based on `docs/ai-use-log.md`.

10. Limitations

Current limitations:

- The local subprocess backend is not a security boundary.
- Docker/gVisor checks require the Linux target host; local macOS can run only local/API validation.
- The HumanEval/MBPP subset manifest is planned but not complete at the original stretch-target scale.
- The adversarial suite is not yet at the originally approved >=40-program target.
- gVisor timing evidence needs cold-start versus warm-run separation.
- The HTTP API is intentionally minimal and lacks production auth/TLS/rate limiting.

- The benchmark does not yet include confidence intervals across >=30 samples per final condition.

These limitations reduce the strength of final claims, but they do not erase the project contribution. The final deliverable is a working, inspectable, reproducible sandbox artifact with measured evidence and a clear path for future expansion.

11. Future Work

Future work should proceed in this order:

1. Complete HumanEval/MBPP subset manifest with task IDs, provenance, license notes, expected outputs, and exact scoring policy.
2. Expand adversarial corpus to the planned category and count targets.
3. Add benchmark fields that separate image pull, cold container startup, warm startup, and program runtime.
4. Add confidence intervals and repeated final samples per backend/condition.
5. Add Firecracker or another microVM backend as a third comparison point.
6. Add CI or a dev container for host-side Python 3.11 consistency.
7. Replace the minimal stdlib HTTP server with a production-grade API only if the project moves beyond academic evaluation.

12. Conclusion

SafeExec demonstrates that a small CISC 699 artifact can move beyond a demo and make code-execution sandbox behavior visible. The project produced a compact Python service, local and container backend paths, a repeatable validation harness, Docker/gVisor target-host evidence, reproducibility docs, final evidence transcripts, and a reviewer-facing artifact package.

The final claim is deliberately measured: SafeExec is not a production sandbox and does not prove general adversarial safety. It does show graduate-level technical design, implementation, evaluation discipline, reproducibility, and honest limitation reporting for the problem of LLM-agent Python code execution.

References

[1] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST Trustworthy and Responsible AI Report AI 100-2 E2025, 2025.

[2] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, OWASP Gen AI Security Project, v2025 materials.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," arXiv:2302.12173, 2023.

[4] The gVisor Authors, "Security Model," *gVisor Documentation*.

[5] The Linux Kernel documentation, "Seccomp BPF (SECure COMPuting with filters)."

[6] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The True Cost of Containing: A gVisor Case Study," USENIX HotCloud, 2019.

[7] Red Hat Product Security, "runc - Malicious container escape - CVE-2019-5736," 2019.

[8] GitHub Advisory Database, "runc vulnerable to container breakout through process.cwd trickery and leaked fds - CVE-2024-21626," GHSA-xr7r-f8xq-vfvv, 2024.

[9] A. Agache et al., "Firecracker: Lightweight virtualization for serverless applications," USENIX NSDI, 2020.

[10] OpenAI, "Code Interpreter," OpenAI API Documentation.

[11] Anthropic, "Code execution tool," Claude API Documentation.

[12] E2B, "E2B Documentation."

[13] Modal, "Sandboxes," Modal Documentation.

[14] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," ICLR, 2024.

[15] J. Pineau et al., "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)," arXiv:2003.12206, 2020.

[16] J. Pineau, "The Machine Learning Reproducibility Checklist," v2.0, 2020.

[17] OpenAI, "HumanEval: Hand-Written Evaluation Set," GitHub repository.

[18] Google Research, "Mostly Basic Python Problems Dataset," GitHub repository.

Appendix A. Submitted Evidence Index

Evidence	File
Final evidence summary	`docs/14-hard-stop-7/evidence/final-evidence-summary.txt`
Smoke transcript	`docs/14-hard-stop-7/evidence/smoke-output.txt`
Unit-test transcript	`docs/14-hard-stop-7/evidence/unit-tests-output.txt`
Local validation summary	`docs/14-hard-stop-7/evidence/validation-summary.txt`
API trace	`docs/14-hard-stop-7/evidence/api-trace.json`
Reproducibility audit	`docs/14-hard-stop-7/evidence/reproducibility-audit.md`
Environment snapshot	`docs/14-hard-stop-7/evidence/environment-snapshot.txt`
Docker/gVisor target-host summary	`docs/14-hard-stop-7/evidence/w8-target-midpoint-summary.md`
W10 clean-run transcript	`docs/14-hard-stop-7/evidence/w10-clean-run-output.txt`
Final artifact package	`docs/14-hard-stop-7/safeexec-final-artifact-package.zip`

Appendix B. AI-Use Disclosure Summary

Generative AI assistance was used throughout the project for ideation, drafting, code scaffolding, evidence interpretation, document generation, presentation generation, and command planning. The student retained responsibility for scope decisions, final review, command execution approval, Canvas submission, and all claims. The authoritative audit log is `docs/ai-use-log.md`.

Appendix C. Requirements-to-Evidence Matrix

Requirement / claim	Final evidence	Status
Structured Python execution request and result model	`src/safeexec/models.py`; API trace in `docs/14-hard-stop-7/evidence/api-trace.json`	Implemented
Development smoke path for local execution	`src/safeexec/backends/local.py`; `smoke-output.txt`; local validation 12/12	Implemented with explicit non-security caveat
Hardened Docker command path	`src/safeexec/backends/docker.py`; W8 target-host Docker 50/50	Implemented and target-host validated
gVisor command path using `runsc`	`src/safeexec/backends/docker.py`; W8 target-host gVisor 49/50; `w8-target-runsc-version.txt`	Implemented with timing limitation
JSON API shell for `/health` and `/execute`	`src/safeexec/api/server.py`; final API trace	Implemented
Repeatable validation and evidence capture	`scripts/run_validation.py`, `scripts/run_midpoint_evidence.py`, `scripts/run_final_evidence.py`	Implemented
Reproducible handoff package	`docs/reproducibility/`; `scripts/package_final_submission.py`; final artifact ZIP manifest	Implemented
AI-use disclosure and academic integrity	`docs/ai-use-log.md`; Appendix B	Implemented

Appendix D. Final Package Inventory

The Canvas submission package is intentionally redundant so the grader can review the work either as a polished report/deck or as raw technical evidence. The final report and integrated summary explain the project; the artifact ZIP contains the runnable code and reproducibility materials; the evidence ZIP contains direct logs and structured outputs; the presentation deck supports the recorded walkthrough.

Package item	Purpose
`Final-Technical-Report.pdf` / `.docx`	Main technical argument and final capstone report
`Final-Integrated-Submission.pdf` / `.docx`	Short Canvas-facing inventory and rubric map
`safeexec-final-artifact-package.zip`	Source, tests, scripts, reproducibility docs, logs, evidence, and manifest

`final-evidence.zip`	Raw W14 evidence folder for independent verification
`Final-Presentation-and-Demo-Deck.pptx` / `.pdf`	Final presentation and demo support
`final-presentation-demo-script.md`	Recording script for the required walkthrough/demo
`final-release-notes.md`	Archive/release note describing the submitted final package

Appendix E. Final Walkthrough Checklist

The required recorded walkthrough should show the final repository and execute the key commands rather than only narrating slides. The recommended sequence is:

1. Open the final report and state the problem, artifact, and evidence claim.
2. Show `README.md`, `src/safeexec/`, `scripts/`, `tests/`, and `docs/reproducibility/`.
3. Run or show fresh transcripts for `make smoke`, `make test`, validation, and reproducibility audit.
4. Open the W8 target-host evidence summary and explain the Docker/gVisor boundary.
5. Open `safeexec-final-artifact-package.manifest.json` and explain what is archived.
6. Close with limitations: local backend is not a sandbox, gVisor timing needs refinement, and future work expands corpus depth.